



PDF Download
3724363.3729096.pdf
16 January 2026
Total Citations: 0
Total Downloads: 534

Latest updates: <https://dl.acm.org/doi/10.1145/3724363.3729096>

RESEARCH-ARTICLE

Quack the Code: A Computer Game Show Offers Learning Through Teaching AI in Undergraduate Software Engineering

KATHLEEN MARIE KELLY, Colorado School of Mines, Golden, CO, United States

BO WU, Colorado School of Mines, Golden, CO, United States

CHRISTINE LIEBE, Colorado School of Mines, Golden, CO, United States

Open Access Support provided by:

Colorado School of Mines

Published: 13 June 2025

[Citation in BibTeX format](#)

ITiCSE 2025: Innovation and Technology
in Computer Science Education
June 27 - July 2, 2025
Nijmegen, Netherlands

Conference Sponsors:
SIGCSE

Quack the Code: A Computer Game Show Offers Learning Through Teaching AI in Undergraduate Software Engineering

Kathleen Marie Kelly

kathleenmariekelly@mines.edu

Colorado School of Mines

Golden, CO, USA

Bo Wu

bwu@mines.edu

Colorado School of Mines

Golden, CO, USA

Christine Liebe

cliebe@mines.edu

Colorado School of Mines

Golden, CO, USA

Abstract

Gamification is a popular pedagogical approach in computer science education, and advances in large language models (LLMs) enable dynamic and engaging learning experiences, including competition driven by artificial intelligence (AI). This study introduces Quack the Code, a game-show-themed learning tool where students teach an LLM-powered "debug duck" while competing in an AI-hosted game. AI was used to generate personalized avatars, usernames, questions, host responses, and interactions with game elements. Offered to undergraduate computer science students (N=25) over a 16-week semester, the game was used repeatedly, mainly near quiz and exam periods. Most of the students (87%) preferred non-competitive play and focused on Java Basics, the default topic. The students were divided when choosing the type of question, with 45% requesting the default option of code debugging exercises and 47% requesting multiple choice questions. The students also changed the difficulty of the question from the default setting (44%) to the most difficult setting (37%). Focus groups highlighted positive reactions to competition, AI interactivity, and customization, but noted challenges with unclear instructions and user interfaces. The results suggest that LLM-powered gamification can enhance engagement and learning, with future improvements focused on usability and curriculum integration to increase participation and impact.

CCS Concepts

• **Software and its engineering** → **General programming languages**; • **Social and professional topics** → **Computer science education**.

Keywords

gamification, higher education, knowledge, engagement, games, computer science, game-based learning, feedback, debug duck, large language models, motivation, competition, artificial intelligence

ACM Reference Format:

Kathleen Marie Kelly, Bo Wu, and Christine Liebe. 2025. Quack the Code: A Computer Game Show Offers Learning Through Teaching AI in Undergraduate Software Engineering. In *Proceedings of the 30th ACM Conference on Innovation and Technology in Computer Science Education V. 1 (ITiCSE 2025)*, June 27-July 2, 2025, Nijmegen, Netherlands. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3724363.3729096>

1 Introduction

Instructors continuously seek pedagogical techniques to improve student engagement and motivation in educational settings. Computer science students often struggle with the course content, leading to frustration and decreased motivation [4]. Although gamification has been shown to be effective in addressing this challenge [7], developing games that balance course content with individual student needs remains difficult [22]. Meanwhile, large language models (LLMs) have been shown to be a promising source for generating course content [19]. In this research project, the goal was to increase student motivation by offering a gamified learning tool that could be used throughout the semester. This project explored the usefulness of LLMs in conjunction with the gamification of software engineering content. Combining gamification and generative artificial intelligence (AI) allowed for a more personalized experience for the students, as they had the opportunity to choose much of the game's content and generate a personalized avatar and username. The novelty of this research also included LLMs as competitors in the game itself, as well as the opportunity for students to learn by teaching AI. This combination of gamification and AI added a fun factor and an element of unpredictability.

Specifically, the gamified tool involved students learning while also teaching their AI-powered debug duck. Rubber duck debugging was a technique popularized in 1999 [5] in which a student keeps a rubber duck with them while they work on their code. As the student writes a program, they also tell the duck exactly what they are trying to do on each line of code. By speaking out loud to this inanimate object, the student often discovers and solves any problems with their program. This concept was the main theme of the gamified tool, allowing the student to learn while also teaching their debug duck. The goal was to teach the duck enough content to compete in a final level where only the student's duck would be allowed to answer the questions.

The gamified learning tool was offered to all students in a university software engineering course. This tool was not part of the standard curriculum, but was offered as an additional resource for students



This work is licensed under a Creative Commons Attribution 4.0 International License. *ITiCSE 2025, June 27-July 2, 2025, Nijmegen, Netherlands*
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1567-9/2025/06
<https://doi.org/10.1145/3724363.3729096>

to use on their own time for most of the semester. Specifically, the following research questions (RQs) were investigated:

RQ1: To what extent do students voluntarily engage in an optional gamified learning tool outside of class over a semester?

RQ2: Do students prefer non-competitive or competitive modes when using an AI-powered gamified tool, and what factors influence their choice?

RQ3: How does interaction with an LLM-powered gamified tool correlate with self-reported improvements in software engineering skills?

2 Related Work

Gamification is the idea of adding game elements to areas that would otherwise not be considered a game [3]. For computer science education, this means adding game elements to the curriculum that support the learning of computer science concepts. As LLMs become more popular, games have the ability to be more personalized and customized for every student's needs. Combining gamification and LLMs into the computer science classroom is an area that is still underexplored.

Gamified learning appears to benefit university students in a data structure course more than non-gamified learning. Mellado and Cubillos [12] studied university students in a data structure course with and without gamified learning objectives. The authors found that the gamification techniques "significantly outperformed" the exercises that were not gamified. Gamification is more effective if the individual needs and characteristics of the student are taken into account. Makhovych [11] looked at popular educational game platforms like Kahoot!, Quizlet, Duolingo, and Memrise in an attempt to analyze the key elements of educational gamification. Each platform offered unique features, but Makhovych concluded that "the effectiveness of gamification in education depends on how it is integrated into the overall learning system".

Meeting the needs of individual students has been novelly recorded with the use of generative AI in education. In a literature review by Babu and Moorthy [21], the authors stated that "gamification elements, user profile elements, and AI approaches" are all interconnected with each other. The authors explained that gamification helps improve student motivation, and by adding AI, the game elements can be dynamically adapted for each student. Montella et al. [13] used an LLM for the automatic generation of gamified exercises in a programming course. The authors created GAMAI, a gamified tool that used OpenAI to generate gamified programming exercises for instructors to use in their courses. The novelty was in the storytelling component that teachers could add to the tool, which offered a way for instructors to personalize their exercises. Rekha et al. [20] discussed an AI-powered system that would personalize learning according to the needs of each student. By collecting student data related to learning styles and preferences, the authors used AI to identify patterns and develop a personalized "learning pathway" for each student. Similarly, Ogli [15] explored the idea of integrating AI into computer science learning to create more personalized experiences.

AI and gamification were also studied in conjunction with student attendance. Limonova et al. [10] explained that student engagement was directly related to student attendance. By increasing student engagement with gamification and AI techniques, students would be more likely to attend class and have better success with learning outcomes. Narayanan and Kumaravel [14] reported on students who had the opportunity to earn points for attendance and interaction with an AI-powered interactive learning environment. Instructors would be able to collect data from the system to better identify struggling and high-achieving students and support them both in the most effective way.

The challenge lies in creating a game that would be the most effective for any student and one that teaches the most beneficial content. Dasey [6] noted that games need to teach life skills, putting students in decision-making situations that are relevant to the learning content. AI would be able to produce the situation and also return feedback, giving more targeted learning to the student. According to O'rnbekovna [16], the use of AI with games will make learning less "like a chore and more like a challenge to conquer".

In each of these related works, the authors were looking at ways to use gamification, LLMs, or both to improve student learning outcomes and motivation. Studies looked at using AI to create exercises and experiences for students while also trying to personalize the encounter. In previous research with AI-generated educational games, this author used AI, but in a very limited way and for a very short period of time [9]. In this study, an educational game show was created that allowed the student to choose several parameters of the game, used AI to generate all the game questions and modes, and was available for the entire semester. This method of combining gamification and AI in many different approaches is a novel research study and a gap in research.

3 Methods and Experiment

3.1 Methods

Students in undergraduate software engineering often struggle with object-oriented concepts as well as with debugging code, reading code, and refactoring code. This author, a four-year full-time computer science teaching professor, brainstormed ideas that would reinforce these concepts in a gamified and competitive way. Using AI, some game ideas included having AI generate questions and answers, having a debug duck involved in the game play, having students determine good versus bad code, and having students attempt to outsmart AI. There was also some brainstorming around having students fix buggy code, specifically trying to fix bugs faster than an AI opponent adds them. A real-world scenario could involve the role of an employer accepting or rejecting code based on criteria.

AI was also used as inspiration in the search for an educational computer science game. Some ideas returned from AI included a debug detective game, a debug duel between a human and an AI, a code-defender game between a human and a bug AI, and a code-architecture game. Each idea came with a core concept and then a list of game mechanics around challenges, scoring, leaderboard, time pressure, and others. The final game design chosen for this

project attempted to include many of the ideas gathered, a couple of game mechanics suggested by AI, and then an overall theme of the game.

For this qualitative study, a gamified learning tool was created and deployed on an internal campus Web server. The game, entitled "Quack the Code", was designed with a game show theme where a student and his/her debug duck would compete against an AI opponent and AI debug duck. As contestants answered questions, those answers would also become part of their duck's knowledge base. A "duck showdown" level would allow a student to challenge another student, using only their ducks as the contestants. Depending on how well each student taught their duck would determine the ultimate winner on a leaderboard. The game provided content from over 20 class modules, varying levels of difficulty, varying types of questions, and varying game modes. See Figures 1 and 2 for images of the game splash screen and the introduction screen.

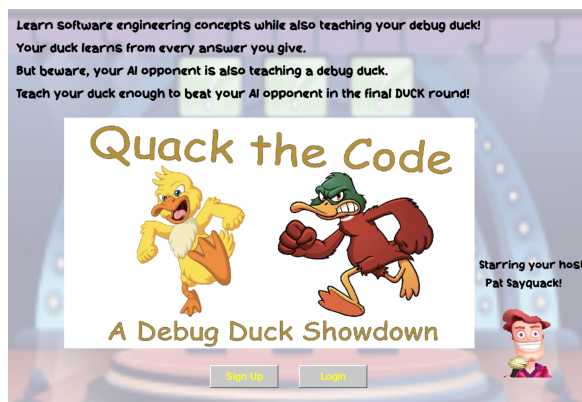


Figure 1: Quack the Code splash screen

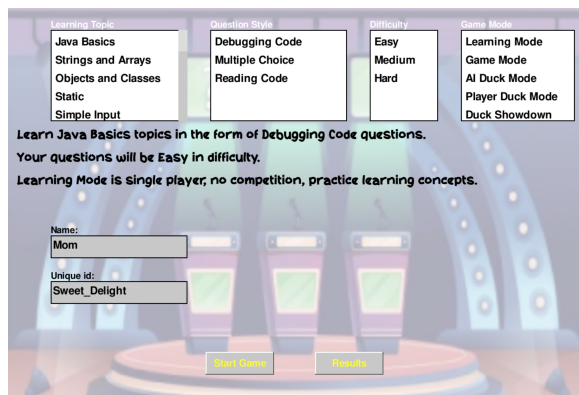


Figure 2: Quack the Code intro screen

The game was created using the python and pygame [17] libraries. OpenAI methods were used to integrate LLM components, with GPT-4-Turbo [2] as the model. Custom avatars were created for students and their ducks using Dalle from Langchain [1]. To deploy the game on a Web server, it was packaged using pybgag [18]. The

components of the user interface communicated with a python flask server [8] to make any calls to the LLM methods, storing anonymous usage data on the flask server in JSON files.

AI was used for the generation of the questions, for the AI opponent, for the debug ducks, and for the game show host. Prompt engineering was used in each situation to generate relevant content, have an AI opponent who was competitive, have a debug duck that could only answer based on its knowledge base, and have a game show host who would keep the contestants focused. For example, in the case of the student requesting code to debug for their game show question, the LLM prompt included the following:

"You are a helpful assistant that generates Java code that contains syntax and logic errors. Generate a snippet of Java code that contains sections related to a given topic. This code should contain syntax and/or logic errors. Explain the problem to the user as a question, also telling the user that this snippet of code contains logic errors, and the user must identify the line numbers with errors. Be sure and give each line of code a line number. Also, respond with the answer to the problem, identifying the line numbers with errors and explanations. Make sure that the code is formatted correctly to make it easier to read."

This prompt was then appended with the user's learning objective and difficulty-level selections. The questions or code returned from AI were relevant to the learning objective and seemed to be a level of difficulty that matched the user's selection. Once the user answered the question, the game show host AI persona would grade it using the AI-generated answer as compared to the user's answer. The host would also offer an explanation of the answer to give the student more context. In the case where the user answered off-topic or with a joke, the host was prompted to become impatient yet still offer the correct answer and explanation. The host of the game show was there to provide learning guidance and also some humor to the students.

When the user chose to play against an AI opponent, the AI opponent was prompted to answer the question using a randomly generated accuracy percentage. The opponent was also instructed to keep their answer to 20 words or less. See Figure 3 for a screenshot of a user playing against an AI opponent. Once the user typed into the answer box and clicked the "Answer" button, the game show host would grade the answer. In this competitive mode of the game, the user's answer would also be used to teach his/her debug duck, being stored in a brain facts file. Similarly, the AI opponent would have the chance to answer the question, receive a grade, and store their answer into their debug duck's brain facts file. In this mode, the student had the opportunity to learn while in a competitive setting but also the opportunity to learn while teaching. The student would realize that any answer given was being identically copied into their debug duck's knowledge base. Therefore, a more specific and well-explained answer would better benefit the debug duck in a final showdown.

When the user chose to play his/her debug duck against an opposing debug duck, the duck was prompted to respond with only the knowledge in the brain facts file. If the duck could not determine the answer from the brain facts file, they were instructed to answer

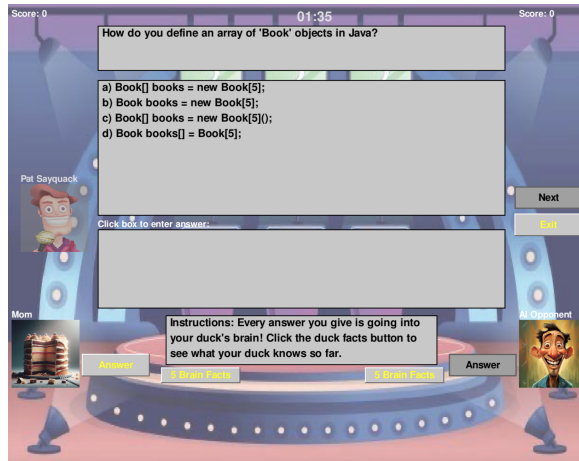


Figure 3: Competing against AI opponent to train ducks

with something funny, such as a joke to add a lighthearted element to the game. In this mode, students were able to observe how well their debug duck had been trained, and then iterate and improve on that training. After a competition, whether against AI or another student, the winner would have the opportunity to claim an AI-generated prize. This prize was generated using prompt engineering that included the contestant's final score. AI was asked to generate a prize description that ranged from fabulous to silly depending on the final score, again to add a lighthearted element. The complete game workflow can be seen in Figure 4.

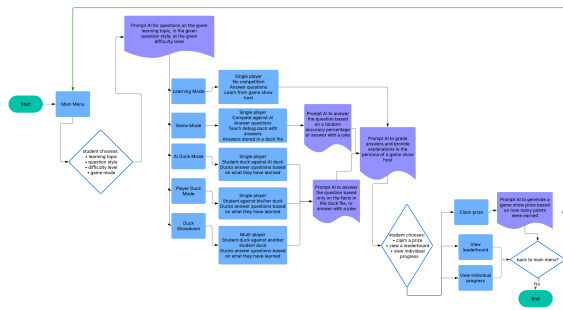


Figure 4: Flowchart of the game workflow

3.2 Experiment

For this research experiment, three sections of an undergraduate software engineering course were given the opportunity to participate in this study (N=116). This course is a required course for computer science majors populated mostly by sophomores. The course covers software engineering practices while programming both small and large software projects in the language of Java. All students received an informed consent form followed by a link to an interest survey. After completion of the survey, students were given access to the game for the remainder of the semester. There was also a voluntary focus group observation day at the end of the semester.

The participating students signed up with an anonymous username followed by an avatar description. This would cause a custom avatar image and a custom unique id to be generated for the student to use on future logins. Once logged in, students' interactions with the game were saved in JSON files on the flask server for later analysis. A student could choose to use the game simply as a learning tool with no competition or as a game show participant. The student would select the question content, difficulty, and whether to receive questions as multiple-choice, as debugging code, or as reading and comprehending code. Points were earned in all modes, giving the student the opportunity to see their ranking in an overall student leaderboard. The results of the competitions would allow the student to win an AI-generated game show prize.

Students also had the opportunity to participate in a voluntary observation day of "Quack the Code". During this half-day session, participants were observed in their interaction with the game and with each other, followed by a short interview.

4 Results

The results of signing the informed consent and completing the student interest survey included responses from 30 students (N=30). The survey included 28 questions with a Likert scale, some questions worded positively while others used reverse coding. The higher scoring results showed that students find programming enjoyable and stimulating, that students would be motivated to learn computer science through the use of a learning tool, and that students would be motivated to learn computer science through the use of a game. The results also showed that students would use a game to learn computer science concepts. In terms of common elements of the game, students enjoy competition, music and sounds, leaderboards, and avatars in the games. Students scored the use of generative AI in games as less enjoyable. Students reported a desire for more help outside of class and more office hours on a larger scale than their struggles with debugging or understanding code. If the students had access to an educational game for class, it was reported that the majority would play the game between "A couple times" and "Often".

The first research question explored whether students would use a learning tool outside of class. The results of this study showed that 22% of the students used the tool at least once. Students were reminded weekly about the tool during the two-month duration of this investigation, but the majority of students did not try it. After interviewing students about their use of the tool, students commented that they did not use it due to lack of time outside of class and the barrier of filling out an informed consent form.

The second research question specifically explored the students who used the learning tool and how they chose to use it. Students who completed the survey were given access to the game for the remainder of the semester. Most of the students had access to the game for 8-10 weeks depending on when the survey was completed. Anonymous usage data was collected throughout the semester and summarized into common usage themes. During the course of two months, 25 students played the game. Analysis showed that the students varied in the number of game sessions played, with 7

students playing the game exactly once, but the other 18 students playing it between 2 and 14 times. See Figure 5.

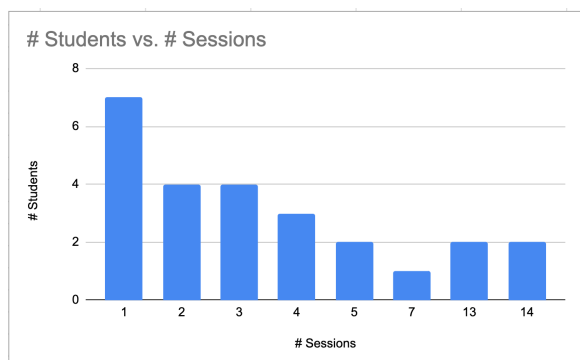


Figure 5: Student Number of Sessions

The vast majority of sessions (87%) were played as a non-competitive learning tool, potentially because Learning Mode was the default game mode on startup. Similarly, the default learning topic was Java Basics, which was also found to be the majority of the selection (52%). The default setting for the difficulty of the game was an easy level, but the students chose varying levels of difficulty in their sessions. The default question type in the game was debugging code questions, a snippet of AI-generated code that contained intentional bugs. Students mostly chose questions of debugging code or multiple choice over questions of reading comprehension. See Figures 6, 7, 8, and 9. The results showed that the default settings of the game had a huge impact on the students' selections, especially for the non-competitive game mode and the subject of the game questions.

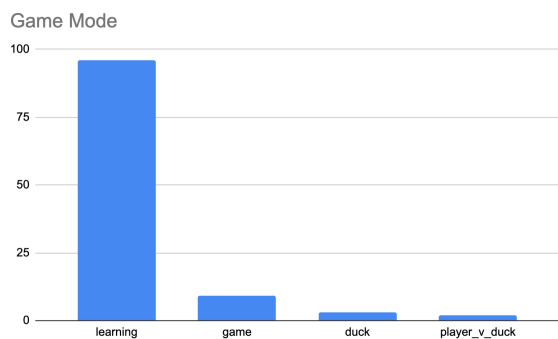


Figure 6: Game Mode Selections

There was also an organized game day on which the students were invited to play while being observed on a particular morning. Nine students participated in this play day and interview session, leading to some interesting quotes and feedback. Students who played in this forum tried various topics, question types, and game modes, then answered interview questions. From observation, the students were not entirely sure what the different game modes were until verbally asked, but then tried several different modes. Once playing,

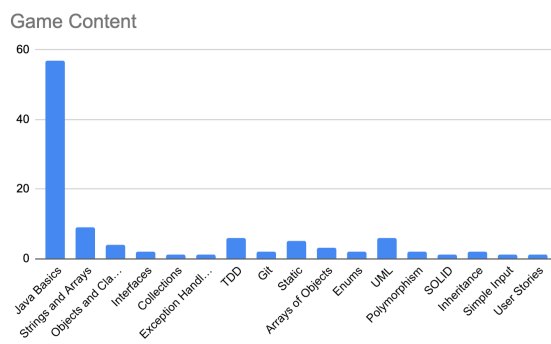


Figure 7: Game Content Selections

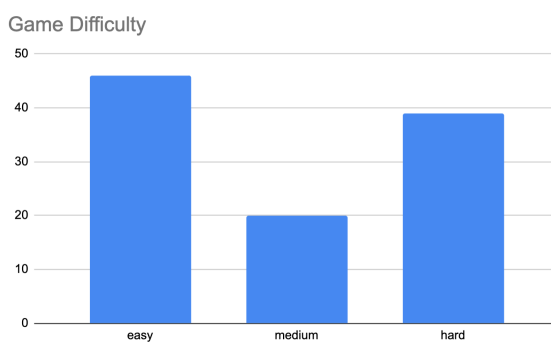


Figure 8: Game Difficulty Selections

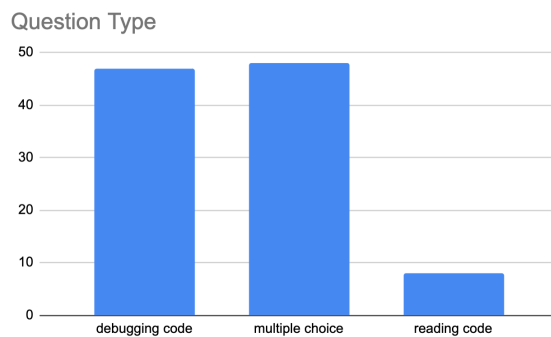


Figure 9: Question Type Selections

there were mixed feelings as to whether playing solo or against a competitor was more enjoyable. Students admitted to either loving competition or preferring to avoid competition. For those who disliked competition, they also commented that using the learning mode helped them be more efficient with their time and helped them "get the learning done" and get "more reps". Other students commented that once they realized that the competitive mode was there, they thought it was very fun and more engaging.

The third research question explored whether the learning tool improved students' abilities in the course. During the day of play,

the students remarked that the use of AI in the game helped their learning, especially when it came to language syntax and logic processing. Students commented:

"Either the AI or host would point out something I missed, something I'm well aware that I tend to do when I skim through code."

"One of the aspects that I found the most useful was that it would point out when I did accidentally miss something small. In these cases it was correct in pointing it out. It seems to be really good at finding syntax mistakes."

Students also enjoyed the game show theme and the customized profiles, commenting:

"I loved the duck theme, I thought it was funny"

"The music is the funniest part."

"I would play the duck versus duck mode in class, not so much human versus human because of impostor syndrome. I liked it a lot, I thought it was really fun. I don't know of anything else like this. It feels more personalized than anything I have seen."

The use of the game started in early September and continued until early November. Of the 110 unique sessions of the game, 59 occurred on the day of the evening midterm exam, showing an increase in tool usage in preparation for the midterm. There was also an increase in usage, about 13 sessions, before a particular quiz in September. **This is an exciting result, as it shows that students would use a learning tool outside of class to help prepare for an upcoming exam or quiz.** In particular, a user of the tool logged in for 90 minutes and switched between six learning concepts and the three types of questions on the day of the midterm, indicating a wide range of practice before an exam. According to this student:

"I used it in the beginning because I had never used Java. It was helpful with the first quiz so I used it for the midterm since I needed something to study with. There was no other learning tool out there and it was really helpful for that."

Another student used it a month after it was introduced, once course concepts started to become more difficult. According to this student:

"I wanted to study for an upcoming quiz. I didn't fully understand the material and wanted to do well."

When asked about AI accuracy and fairness, the students commented that the questions and answers felt fair, although there were a couple times that it was inaccurate with an answer or was unclear with what the question was asking. Several students attempted to steer the AI responses off topic. When asked a particular software engineering question, one user responded by telling the AI to ignore the question and instead to give that student a script related to a completely different course assignment. Due to the prompt engineering of the AI game show host, the AI responded in a scolding tone, reminding the user to stay on topic and not ask for inappropriate content. **This was an exciting result, as it showed the ethical power of the GPT model, as well as the effect of prompt engineering on the AI persona.**

5 Conclusion and Future Work

This research project brought together two novel pedagogical approaches: gamification and the use of generative AI in the classroom. This author combined these two approaches in the creation of a novel game, "Quack the Code". In this game, students would compete in a game show atmosphere, learning software engineering concepts while also training their debug duck for future rounds. The goal was to provide a gamified learning tool to students in an undergraduate software engineering course in hopes of observing them utilizing the tool in a gamified way with each other both in and outside of class. The results of this study showed the usage of the game, but not from as many of the students as hoped. However, the results showed an increase in usage around assessment days and positive feedback during a focus group.

There are several areas for future work with this project. Specifically with "Quack the Code", future work would include changes to the user interface, the default settings, and a clearer explanation of the game to eliminate some of the elements of confusion. Once the game was demonstrated in class, the students seemed much more aware of its purpose and how it would be used. Some of the game components and instructions were unclear. The consent form was noticed by several students as a barrier to playing the game.

As this was an exploratory study to understand the usage of games outside of class, there was no baseline comparison to traditional teaching tools. Future work would involve a control group using traditional learning tools to compare with the group using this gamified tool. In addition, a future semester would make the gamified learning tool part of the regular course curriculum. The students who used the game suggested a tournament-style event that could take place during class.

Based on the usage of "Quack the Code" throughout the semester and on the observation day, there are some exciting directions this research could take. One of the drawbacks of this project was the time it took to iterate. Creating the game and then presenting it to students for the semester took more than three months. Gamification has been researched as working, but limits instructors by the game itself. An idea for future work would be to give students a much larger role in the creation of the game. Creating this learning tool would be part of the curriculum, and the students would work together to build a game that they considered to be motivating and helpful. By combining gamification, generative AI, and the students' own novel ideas, a perfect blend could be created to inform the field of computer science.

6 Acknowledgments

The author thanks Dr. Bo Wu for advice on the direction of this research, tying together the novel use of large-language models with her interest in gamification of computer science materials. She also thanks Dr. Christine Liebe for advice on how to research a topic of interest and how to get student involvement in her research. She also thanks the members of the Pygame Community Discord channel who offered advice related to pygbag. Lastly, she thanks her teaching assistants for their initial feedback on the game and all her students who volunteered to help with her research this semester by playing the game.

References

- [1] 2024. *Dall-E*. https://python.langchain.com/docs/integrations/tools/dalle_image_generator/
- [2] 2024. *OpenAI*. <https://openai.com/index/gpt-4/>
- [3] Inigo Aldular. 2025. Enhancing software development education through gamification and experiential learning with genially. *Software Qual J* 3 (2025).
- [4] Shannon Butler and Dewan Tanvir Ahmed. 2016. Gamification to engage and motivate students to achieve computer science learning goals. In *2016 International Conference on Computational Science and Computational Intelligence (CSCI)*. IEEE, 237–240.
- [5] Kolade Chris. 2022. Rubber Duck Debugging. *freecodecamp* (2022). <https://www.freecodecamp.org/news/rubber-duck-debugging/>
- [6] Tim Dasey. 2024. 'Wisdom Skills' Are Hard to Teach - AI Can Help. *Inside Higher Ed* (2024).
- [7] Mourya Reddy Narasareddy Gari, Gursimran Singh Walia, and Alex David Radermacher. 2018. Gamification in computer science education: A systematic literature review. In *2018 ASEE Annual Conference & Exposition*.
- [8] Miguel Grinberg. 2018. *Flask web development: developing web applications with python*. "O'Reilly Media, Inc."
- [9] Kathleen Kelly, Bo Wu, and Christine Liebe. 2024. Gamification Powered by a Large Language Model to Enhance Flipped Classroom Learning in Undergraduate Computer Science. *European Conference on Games Based Learning* 18 (10 2024), 930–937.
- [10] Viktoriya Limonova, Arnaldo Manuel Pinto dos Santos, José Henrique Pereira São Mamede, and Vitor Manuel de Jesus Filipe. 2024. Maximising Attendance in Higher Education: How AI and Gamification Strategies Can Boost Student Engagement and Participation. In *Good Practices and New Perspectives in Information Systems and Technologies*, Álvaro Rocha, Hojjat Adeli, Gintautas Dzemyda, Fernando Moreira, and Aneta Poniszewska-Marañda (Eds.). Springer Nature Switzerland, Cham, 64–70.
- [11] Inna Makhovych. 2024. Gamification: Individualized Learning Aimed at Enhancing Motivation Among Computer Science Students in the English Language Classroom. (07 2024).
- [12] Rafael Mellado and Claudio Cubillos. 2024. Gamification improves learning: Experience in a training activity of computer programming in higher education. *Journal of Computer Assisted Learning* 40, 4 (2024), 1959–1973. arXiv:<https://onlinelibrary.wiley.com/doi/pdf/10.1111/jcal.13000> <https://onlinelibrary.wiley.com/doi/abs/10.1111/jcal.13000>
- [13] Raffaele Montella, Ciro Giuseppe De Vita, Gennaro Mellone, Tullio Ciricillo, Dario Caramiello, Diana Di Luccio, Sokol Kosta, Robertas Damasevicius, Rytis Maskeliunas, Ricardo Queiros, and Jakub Swacha. 2024. GAMAI, an AI-Powered Programming Exercise Gamifier Tool. In *Artificial Intelligence in Education. Posters and Late Breaking Results, Workshops and Tutorials, Industry and Innovation Tracks, Practitioners, Doctoral Consortium and Blue Sky*, Andrew M. Olney, Irene-Angelica Chounta, Zitao Liu, Olga C. Santos, and Ig Ibert Bittencourt (Eds.). Springer Nature Switzerland, Cham, 485–493.
- [14] K Sankara Narayanan and A. Kumaravel. 2024. Hybrid Gamification and AI Tutoring Framework using Machine Learning and Adaptive Neuro-Fuzzy Inference System. *Journal of Advanced Research in Applied Sciences and Engineering Technology* 42, 2 (Apr. 2024), 221–233. https://semarakilmu.com.my/journals/index.php/applied_sciences_eng_tech/article/view/4419
- [15] Nomozov Nurbek Bakhtiyar Oglu. 2024. Improving the Methodology of Teaching Computer Science to Students with the Help of Artificial Intelligence Capabilities. In *Science and innovation*. 383–385.
- [16] Ubaydullayeva Muattar O'rinbekovna et al. 2024. The Use of AI-Powered Technologies in Education. *Innovative Developments and Research in Education* 3, 28 (2024), 224–227.
- [17] pygame. 2020. . <https://www.pygame.org/docs/>
- [18] PyPI. [n. d.]. . pypi.org/project/pygbag
- [19] JD Zamfirescu-Pereira Laryn Qi, Bjoern Hartmann, and John DeNero Narges Norouzi. 2023. Conversational Programming with LLM-Powered Interactive Support in an Introductory Computer Science Course. (2023).
- [20] K. Rekha, Kumaraguruparan Gopal, D Satheeskumar, U. Albert Anand, D. Samuel Sundar Doss, and Shanmuganath Elayaperumal. 2024. Ai-Powered Personalized Learning System Design: Student Engagement And Performance Tracking System. In *2024 4th International Conference on Advance Computing and Innovative Technologies in Engineering (ICACITE)*. 1125–1130.
- [21] Srimathi Suresh Babu and Anitha Dhakshina Moorthy. 2024. Application of artificial intelligence in adaptation of gamification in education: A literature review. *Computer Applications in Engineering Education* 32, 1 (2024), e22683. arXiv:<https://onlinelibrary.wiley.com/doi/pdf/10.1002/cae.22683> <https://onlinelibrary.wiley.com/doi/abs/10.1002/cae.22683>
- [22] Bhisaji C Surve and Arjun P Ghatule. 2024. Gamification empowered with AI tools to enhance student learning engagement and involvement for personalized effective learning experiences. In *ITM Web of Conferences*, Vol. 68. EDP Sciences, 01023.